

AWHDL チュートリアル

Agentic Workflow Harness Description Language v0.1

AI Conductor プロジェクト

2026 年 9 月 25 日

概要

AWHDL は、複数の AI・MCP ツール・決定的プログラム・人間を組み合わせた反復ループを、型付き・イベント駆動の設計として記述する言語です。本チュートリアルでは、実機で動作を確認した 6 つの設計例を順にたどり、**entity** / **architecture**、**device** と **route** の束縛、**signal** と **delta cycle**、タイマーと予算、フィードバックループ、並列実行とバリア、情報フロー検査（暗黙のフローと機密解除を含む）までを学びます。最後に、設計を構成図・振る舞いの図・アクティビティ図・Petri ネット・情報フロー図として描く **aic graph** を紹介します。

目次

1	はじめに	3
1.1	この文書の位置付け	3
1.2	AWHDL が解く問題	3
1.3	全体の構成	3
2	準備	4
2.1	必要なもの	4
2.2	ビルド	4
2.3	検査と実行のコマンド	4
3	最初の設計：entity と architecture	5
3.1	設計	5
3.2	実行	6
4	決定的な判定：deterministic device と if	6
4.1	入力の渡し方	7
5	delta cycle、タイマー、予算	8
5.1	delta cycle	8
5.2	タイマーと予算	8
6	フィードバックループ	9
7	並列実行とバリア	11

8	情報フローと安全	12
8.1	機密区分と配置	12
8.2	さらに厳しくする	13
8.3	暗黙のフロー	13
8.4	機密解除：declassify	13
8.5	設計の外にある安全機構	15
9	設計を図にする (aic graph)	16
9.1	ビューと出力形式	16
9.2	構成を見る：structure ビュー	17
9.3	振る舞いを見る：behavior ビュー	17
9.4	UML アクティビティ図：activity ビュー	19
9.5	Petri ネット：petri ビュー	20
9.6	情報フローを見る：security ビュー	20
10	device と route	24
10.1	束縛の規則	24
10.2	AI Conductor の route (2026 年 9 月時点)	24
10.3	呼び出しと引数	24
11	v0.1 で使えないもの	25
付録 A	文法の早見表	25
付録 B	診断コード	26
付録 C	参考文献	27

1 はじめに

1.1 この文書の位置付け

本書は非規範の入門書です。v0.1 の実装範囲は docs/PROFILE_v0.1_ja.md だけが定め、言語・実行・安全の規則は LANGUAGE_SPEC、RUNTIME_SPEC、SECURITY_SPEC、IR_SPEC が定めます。本書の記述がそれらと食い違う場合は、規範文書が優先します。

本書の例はすべて examples/tutorial/ にあり、2026 年 9 月 24 日に次の 2 つで確認しました。

- aic check による構文・構造・情報フローの静的検査
- aiconductor run-design による実機での実行 (27B モデル、Prolog、Lean、KDB を使用)

第 9 節の図は、2026 年 9 月 25 日に aic graph と Graphviz 2.43 で作成しました。

1.2 AWHDL が解く問題

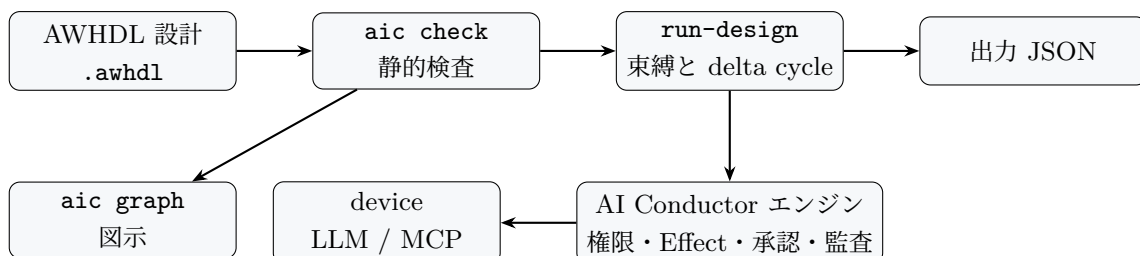
AI にツールを使わせる仕組みでは、「どの AI がどのデータを受け取るか」「いつ再試行し、いつ止めるか」「何をもって完了とするか」がプロンプトの中に埋もれがちです。AWHDL はこれらを設計として書き出し、実行前に検査できるようにします。

AWHDL の基本的な考え方

- AI やツールは **device** (入出力を持つ部品) として扱う。
- データや状態は **signal** が運ぶ。
- **process** はイベントに反応して動く。
- 実行を続けてよいか、外部へ送ってよいか、完了したかは、AI ではなく **AI Conductor ランタイム**が宣言された規則に従って決める。

構文は VHDL に似ていますが、AWHDL はハードウェアを記述する言語ではありません。名前の H は Harness (複数の実行主体を一つの制御構造に束ねるもの) を表します。

1.3 全体の構成



run-design は設計を検査・束縛したうえで delta cycle として実行します。device への呼び出しは、通常の実行と同じエンジンを通るため、権限 (capability) の検査、Effect の記録、承認、監査ログがすべて適用されます。

2 準備

2.1 必要なもの

- Rust (2024 edition) と Cargo
- awhdl リポジトリ (本書の例とランタイムを含む)
- 実行 (run-design) には、AI Conductor のプロジェクトルートと、そこで設定された route (モデルや MCP サーバー)

静的検査 (aic check) と図示 (aic graph) だけなら、モデルや MCP サーバーは不要です。図を画像にするには Graphviz (dot コマンド) を使います。アクティビティ図を PlantUML で描く場合は Java と PlantUML も使います (第 9 節)。

2.2 ビルド

```
cd awhdl
cargo build -p conductor-cli -p aiconductor
# aic          : 構文と静的検査
# aiconductor : AI Conductor ランタイム (run-design を含む)
```

AI Conductor のワークスペースでは、ビルド成果物を OneDrive の外に置くため、次の設定を使います。

```
export CARGO_TARGET_DIR=~/.cache/aiconductor/cargo-target
```

以下では実行ファイルを aic、aiconductor と略記します。

2.3 検査と実行のコマンド

```
# 静的検査 (終了状態 0 = 合格)
aic check examples/tutorial/01_hello.awhdl
aic check examples/tutorial/01_hello.awhdl --output json

# 図示 (DOT を Graphviz で PDF にする)
aic graph examples/tutorial/05_parallel_barrier.awhdl \
    --view behavior --format dot | dot -Tpdf -o behavior.pdf

# 実行 (--project-root は AI Conductor のルート)
aiconductor --project-root <AIConductor> run-design \
    awhdl/examples/tutorial/01_hello.awhdl --input 'task=...'
```

--input 名前=値 は入力ポートに値を与えます。値が JSON として解釈できれば JSON、そうでなければ文字列になります。複数のポートには --input を繰り返します。

表 1 aic check の終了状態

状態	意味
0	実装済みの検査に合格
1	ソースファイルを読み取れない
2	構文解析に失敗
3	構造または情報フローの検査に失敗

3 最初の設計：entity と architecture

3.1 設計

Listing 1 examples/tutorial/01_hello.awhdl

```

1  -- 01: one entity, one architecture, one device, one process.
2  entity hello is
3      port (
4          task    : in  text<internal>;
5          answer  : out text<internal>
6      );
7  end hello;
8
9  architecture behavioral of hello is
10     -- An agent device bound to the configured "initial_coding" model route.
11     device coder : agent generic (location => local, route => "initial_coding");
12 begin
13     process(task)
14     begin
15         coder.run(task) -> answer;
16     end process;
17 end behavioral;

```

entity 設計の外部インターフェースです。port に入力 (in) と出力 (out) を並べます。

型 text<internal> 型名の後の <...> は**機密区分**です。public、internal、restricted の 3 段階があり、第 8 節の情報フロー検査に使われます。

architecture entity の中身です。is と begin の間に宣言を、begin と end の間に process を書きます。

device AI やツールを表す部品です。種類は agent、mcp、deterministic の 3 つです。generic で、どの route (AI Conductor の設定にある実行経路) に結び付けるか、どこで動くか (location) を宣言します。

process(task) 括弧内は**感度リスト**です。task が変化すると本体が実行されます。

coder.run(task) -> answer; device を呼び出し、結果を answer に書き込みます。agent device の場合、メソッド名 (ここでは run) は結果に影響しません。

3.2 実行

```
aiconductor --project-root . run-design awhdl/examples/tutorial/01_hello.awhdl \  
--input 'task=Rustで2つのi32を安全に足す関数を1つだけ書いてください。説明は1行で.'
```

Listing 2 実行結果 (27B モデル、約 4 秒)

```
{  
  "status": "quiescent",  
  "reason": null,  
  "outputs": {  
    "answer": "``rust\nfn safe_add(a: i32, b: i32) -> Option<i32> {\n    a.checked_add(  
    b)\n}\n``\n\nchecked_add` はオーバーフロー時に `None` を返すため、..."  
  },  
  "delta_cycles": 2,  
  "device_calls": 1  
}
```

`status quiescent` は、処理すべきイベントがなくなって自然に終わったことを表します。予算で打ち切られた場合は `exhausted` になり、`reason` に理由が入ります。

`outputs` 出力ポートの最終値です。

`delta_cycles` 実行した `delta cycle` の数です (第 5 節)。

`device_calls` `device` を呼び出した回数です。

4 決定的な判定: deterministic device と if

モデルの出力をそのまま「合格」とみなすと、モデルの誤りがそのまま結論になります。AWHDL では、ツールが返した構造化データを使って、判定を設計の中で決定的に行えます。

Listing 3 examples/tutorial/02_prolog_verdict.awhdl

```
1  -- 02: a deterministic device and a deterministic verdict on its result.  
2  entity prolog_verdict is  
3    port (  
4      task      : in  text<internal>;  
5      verdict   : out text<internal>  
6    );  
7  end prolog_verdict;  
8  
9  architecture checked of prolog_verdict is  
10    device rules : deterministic generic (location => local, route => "logic_rules");  
11    signal result : data<internal>;  
12  begin  
13    process(task)  
14    begin  
15      rules.run_prolog(task) timeout 1 min -> result;  
16    end process;
```

```

17
18     -- The verdict is computed by AWHDL, not by a model.
19     process(result)
20     begin
21         if result.structured.solution_count > 0 then
22             verdict <= "holds";
23         else
24             verdict <= "does not hold";
25         end if;
26     end process;
27
28     process(rules.failed)
29     begin
30         verdict <= "prolog rejected the input";
31     end process;
32 end checked;

```

deterministic 読み取り専用の MCP route にだけ結び付けられる device です。書き込みを伴う route (MATLAB や KDB など) には `mcp` を使います。

signal result : data<internal>; 設計内部の信号です。MCP device の結果は JSON オブジェクトで、`result.text` (ツールの出力文字列) や `result.structured` (ツールが返した JSON) としてフィールドを読めます。

timeout 1 min 呼び出しの制限時間です。時間の単位は `ms`、`sec`、`min`、`hour`、`day` です。

if ... elsif ... else ... end if; 条件分岐です。

rules.failed 呼び出しが失敗したときに起きるイベントです。成功時は `rules.done`、制限時間の超過時は `rules.timeout` と `rules.failed` が起きます。

4.1 入力の渡し方

v0.1 の式にはオブジェクトを組み立てる構文がないため、device に渡すのは主に 1 つの文字列です。ツールの引数 (Prolog のプログラムと問い合わせなど) は、ランタイムのアダプタが入力文字列から取り出します。Prolog では、バッククォートで囲んだ節の並びをプログラム、最後の単一のゴールを問い合わせとして扱います。

```

aiconductor --project-root . run-design awhdl/examples/tutorial/02_prolog_verdict.awhdl
\
--input 'task=`parent(tom,bob). parent(bob,ann). grandparent(X,Z) :- parent(X,Y),
parent(Y,Z).` `grandparent(tom,ann).`'
# => "verdict": "holds"          (delta_cycles 3, device_calls 1)

aiconductor --project-root . run-design awhdl/examples/tutorial/02_prolog_verdict.awhdl
\
--input 'task=`parent(tom,bob). grandparent(X,Z) :- parent(X,Y), parent(Y,Z).` `
grandparent(tom,ann).`'
# => "verdict": "does not hold"

```

ポイント

「成り立つかどうか」は Prolog が返した `solution_count` で決まり、モデルは関与しません。モデルに判断させるのは、決定的に判定できない部分だけにするのが AWHDL での基本的な設計です。

5 delta cycle、タイマー、予算

5.1 delta cycle

AWHDL の実行は **delta cycle** の繰り返しです。1 つの delta cycle では次の順に処理します。

1. 直前に起きたイベントに感度を持つ process を選ぶ。
2. 選んだ process をすべて、同じ時点の値のスナップショットに対して実行する。process 内の代入 (`<=`) や device の結果は、すぐには反映されない。
3. delta cycle の終わりに、書き込みをまとめて反映する。値が**実際に変わった** signal だけが次のイベント (**名前** と **名前.changed**) を起こす。
4. 同じ delta cycle で 2 つの process が同じ signal に異なる値を書くとエラーになる。
5. `assert always` / `assert never` を評価する。

イベントが残らず、タイマーもなければ、実行は `quiescent` で終わります。

5.2 タイマーと予算

Listing 4 examples/tutorial/03_timer_budget.awhdl

```
1  -- 03: delta cycles driven by a timer, bounded by a budget. No devices.
2  entity ticker is
3      port (
4          ticks : out data<internal>
5      );
6  end ticker;
7
8  architecture counting of ticker is
9      timer tick : period 1 sec;
10     signal count : data<internal> := 0;
11     budget run_budget is
12         wall_time <= 3 sec;
13     end budget;
14 begin
15     process(tick)
16     begin
17         count <= count + 1;
18     end process;
19
20     process(count)
21     begin
```

```

22     ticks <= count;
23     end process;
24
25     assert always (count <= 10);
26 end counting;

```

timer tick : period 1 sec; 1 秒ごとに tick イベントを起こします。タイマーを使う設計には、iterations か wall_time の予算が必須です。

budget 実行の上限です。v0.1 で使える項目は iterations (delta cycle 数)、wall_time (経過時間)、tool_calls、model_calls です。

signal count : data<internal> := 0; 初期値付きの signal です。初期値は定数に限ります。

assert always (count <= 10); すべての delta cycle の終わりに成り立つべき条件です。破られると実行は失敗します。

Listing 5 実行結果（デバイスなし、約 3 秒）

```

{ "status": "exhausted", "reason": "wall_time budget",
  "outputs": { "ticks": 2 }, "delta_cycles": 6, "device_calls": 0 }

```

設計の予算とランタイムの予算

設計の budget は、AI Conductor の config/runtime.toml ([loop] 節) の上限をさらに狭めるだけで、広げることはできません。例えばランタイムの max_iterations が 8 のとき、設計で iterations <= 20 と書いても 8 delta cycle で打ち切られます。タイマー間隔を 500ms にすると、この例は設計の wall_time より先にランタイムの上限 (iteration budget exhausted) で止まります。

6 フィードバックループ

検証して、結果によって再実行する。AWHDL では、このループをイベントの連鎖として書きます。

Listing 6 examples/tutorial/04_feedback_loop.awhdl

```

1  -- 04: a feedback loop that repeats a device call until a condition holds.
2  entity kdb_loop is
3      port (
4          code    : in text<internal>;
5          rounds  : out data<internal>;
6          last    : out data<internal>
7      );
8  end kdb_loop;
9
10 architecture feedback of kdb_loop is
11     device q : mcp generic (location => local, route => "table_analysis");
12     signal round : data<internal> := 0;
13     signal result : data<internal>;

```

```

14     budget loop_budget is
15         iterations <= 20;
16         tool_calls <= 5;
17     end budget;
18 begin
19     process(code)
20     begin
21         round <= 1;
22     end process;
23
24     -- Each change of round runs one more call, until three rounds are done.
25     process(round)
26     begin
27         if round >= 1 and round <= 3 then
28             q.run_q(code) timeout 1 min -> result;
29         end if;
30     end process;
31
32     -- q.done fires after every successful call, even when the result is
33     -- unchanged; a signal event would fire only when its value changes.
34     process(q.done)
35     begin
36         last <= result.text;
37         rounds <= round;
38         round <= round + 1;
39     end process;
40
41     assert never (round > 4);
42 end feedback;

```

1. code が与えられると round <= 1。
2. round が変わるたびに、条件を満たせば KDB/q を呼ぶ。
3. 呼び出しが成功すると q.done が起き、round を 1 増やす。
4. round が 4 になると呼び出さなくなり、イベントが尽きて終わる。

signal のイベントと device のイベント

signal のイベントは値が**変わったとき**だけ起きます。同じ計算を繰り返すと結果が変わらないため、process(result) ではループが 1 回で止まってしまいます。呼び出しの完了ごとに必ず起きる q.done を使うのはこのためです。

```

aiconductor --project-root . run-design awhdl/examples/tutorial/04_feedback_loop.awhdl \
--input 'code=sum til 101'
# => "rounds": 3, "last": "{ \"ok\": true, ... \"stdout\": \"5050\\n\" ... }"
#     status quiescent, delta_cycles 8, device_calls 3

```

tool_calls <= 5 は、万一条件を書き誤っても呼び出しが 5 回を超えないための安全策です。assert never (round > 4); は「このループは 4 回目以降に進まない」という設計者の意図を、実

行時に検査できる形で残します。

7 並列実行とバリア

独立した検査は同時に始め、両方がそろった時点で次へ進みたいことがあります。

Listing 7 examples/tutorial/05_parallel_barrier.awhdl

```
1  -- 05: two checks started together; a barrier fires when both have results.
2  entity two_checks is
3      port (
4          rules_task : in  text<internal>;
5          proof_task : in  text<internal>;
6          summary    : out text<internal>
7      );
8  end two_checks;
9
10 architecture joined of two_checks is
11     device rules : deterministic generic (location => local, route => "logic_rules");
12     device prover : deterministic generic (location => local, route => "logic_proof"
13 );
14     signal rule_result, proof_result : data<internal>;
15     barrier both (rule_result, proof_result);
16 begin
17     process(proof_task)
18     begin
19         parallel
20             rules.run_prolog(rules_task) -> rule_result;
21             prover.check_lean_code(proof_task) -> proof_result;
22         end parallel;
23     end process;
24
25     process(both.ready)
26     begin
27         summary <= "rules and proof both returned";
28     end process;
29
30     process(prover.failed)
31     begin
32         summary <= "the proof was rejected";
33     end process;
34 end joined;
```

`parallel ... end parallel;` 中の呼び出しは論理的に並列で、結果は同じ delta cycle の終わりにまとめて反映されます (v0.1 のランタイムは実際には 1 つずつ順に呼び出します)。2 つの分岐が同じ signal に書き込むことはできません (E215)。

`barrier both (rule_result, proof_result);` 列挙した signal がすべて書き込まれると `both.ready` が起きます。

```

aiconductor --project-root . run-design awhdl/examples/tutorial/05_parallel_barrier.
    awhdl \
    --input 'rules_task=`edge(a,b). path(X,Y) :- edge(X,Y).` `path(a,b).`' \
    --input 'proof_task=theorem two : 1 + 1 = 2 := by rfl'
# => "summary": "rules and proof both returned"      (device_calls 2)

# 誤った定理を与えると Lean が拒否し、proof_result は書き込まれない
    --input 'proof_task=theorem bad : 2 + 2 = 5 := by rfl'
# => "summary": "the proof was rejected"

```

失敗した呼び出しは出力 signal に書き込まないため、バリアは成立しません。失敗は `prover.failed` で別に扱います。

8 情報フローと安全

8.1 機密区分と配置

signal とポートの型には機密区分 (`public < internal < restricted`) を付けます。区分のない型は `restricted` として扱われます。device は location (`local` または `cloud`) を宣言し、route の実際の配置と一致しなければ束縛に失敗します。

Listing 8 examples/tutorial/06_information_flow.awhdl

```

1  -- 06: restricted data must never reach a cloud device. aic check rejects this.
2  entity leak is
3      port (
4          secret_notes : in  text<restricted>;
5          advice       : out text<restricted>
6      );
7  end leak;
8
9  architecture unsafe of leak is
10     device reviewer : agent generic (location => cloud, route => "deep_reasoning");
11 begin
12     process(secret_notes)
13     begin
14         reviewer.codex(secret_notes) -> advice;
15     end process;
16 end unsafe;

```

```

$ aic check awhdl/examples/tutorial/06_information_flow.awhdl
awhdl/examples/tutorial/06_information_flow.awhdl:14:9: AWHDL-E301: Restricted data
    cannot flow to cloud device reviewer
$ echo $?
3

```

`restricted` のデータがクラウドの device に流れる経路は、実行前に拒否されます。ランタイムも呼び出しの直前に同じ検査を行うため、静的検査をすり抜けた場合でも送信されません。

8.2 さらに厳しくする

- `assert never (internal -> cloud);` を書くと、`internal` 以上のデータもクラウドへ送れなくなります。
- `device` に `clearance => public` などをつけると、その `device` が受け取れる区分の上限を決められます (超えると E302)。
- 低い区分の `signal` に高い区分のデータを代入することはできません (E303)。区分を下げられるのは、第 8.4 節の `declassify` 文だけです。

8.3 暗黙のフロー

データを直接渡さなくても、情報は漏れます。次の例では、クラウドへ送るのは `public` の `count` だけですが、「送ったかどうか」が `restricted` の `secret` の値を表します。

```
1 if secret = "yes" then
2     reviewer.review(count) -> answer;    -- E305: restricted の条件の中でクラウドを呼ぶ
3 end if;
```

`aic check` は、各文に**文脈の区分**を与えて検査します。文脈の区分は、`process` を起動した事象 (`signal` の区分、`device` の呼び出しに渡したデータの区分など) と、囲んでいる `if` の条件のうち最も強い区分です。文脈の区分が書き込み先より強いと E304、クラウドへの呼び出しなら E305、`device` の `clearance` を超えると E306 になります。

8.4 機密解除：declassify

ローカルの LLM で個人情報を匿名化してから、クラウドのフロンティア AI に解析させたいことがあります。しかし LLM の出力は、漏れていないことを保証できません。そのため、匿名化した結果も `restricted` のままです。区分を下げられるのは `declassify` 文だけで、それには **declassifier** を使います。

`declassifier` は、解除できる範囲 (`from` から `to`) と、解除を認める手段を宣言します。手段は次から構文で選び、両方を書くと両方が必要になります。

- `filter => <device>`, `filter_method => <method>`: ローカルの `deterministic device` による決定的な検査。結果が JSON で `"pass": true` のときだけ合格です。
- `approval => human`: 人の承認。承認者は解除する内容そのものと、どこで解除されるかを見て判断します。

失敗・拒否・承認者の未設定のいずれでも、何も解除しません (fail closed)。

Listing 9 examples/secure_cooperation.awhdl (公式サンプル)

```
1 -- Secure cooperation between a local LLM and a frontier AI in the cloud.
2 --
3 -- The raw records are restricted and never leave the local, protected region.
4 -- A local LLM anonymizes them, but its output is still restricted: an LLM cannot
```

```

5  -- prove that nothing leaked. Only the declassifier `release` may lower it, and
6  -- only when both trusted means allow it:
7  --   (b) a deterministic check on the exact text (filter => pii_check), and
8  --   (a) a human who sees the exact text and where it is released (approval => human
9  --   ).
10 -- The released text is internal, so the frontier model in the cloud may analyze it.
11 --
12 -- `aic check` accepts this design; `aic graph` draws it with the local/cloud lanes.
13 -- At run time the check route must answer {"pass": true} only for text without
14 -- personal data. Until such a route is configured, every release is refused and
15 -- nothing reaches the cloud (fail closed).
16 entity secure_cooperation is
17   port (
18     records  : in  text<restricted>;    -- raw records: never leave the local
19     region
20     question : in  text<public>;         -- what to ask the frontier model
21     analysis  : out text<internal>;       -- the frontier model's answer
22     status    : out text<restricted>     -- why nothing was analyzed, if so
23   );
24 end secure_cooperation;
25
26 architecture hybrid of secure_cooperation is
27   -- Local, protected region
28   device anonymizer : agent generic (location => local, clearance => restricted,
29   route => "initial_coding");
30   device pii_check   : deterministic generic (location => local, route => "
31   text_filtering");
32   device release     : declassifier generic (from => restricted, to => internal,
33   filter => pii_check, filter_method =>
34   run_filter_pipeline,
35   approval => human);
36
37   -- External region
38   device frontier    : agent generic (location => cloud, clearance => internal,
39   route => "deep_reasoning");
40
41   signal draft       : text<restricted>;    -- anonymized by the local LLM, not yet
42   trusted
43   signal anonymous    : text<internal>;      -- released: checked and approved
44
45   budget cooperation is
46     model_calls <= 4;
47     wall_time <= 30 min;
48   end budget;
49 begin
50   -- 1. Anonymize locally.
51   process(records)
52   begin
53     anonymizer.run(records) timeout 5 min -> draft;
54   end process;

```

```

47
48   -- 2. Release only what the PII check passes and a human approves.
49   process(draft)
50   begin
51       anonymous <= declassify draft using release;
52   end process;
53
54   -- 3. Analyze in the cloud. Only internal and public data reach this point.
55   process(anonymous, question)
56   begin
57       frontier.analyze(anonymous, question) timeout 10 min -> analysis;
58   end process;
59
60   -- A refused release keeps everything local.
61   process(release.failed)
62   begin
63       status <= "release refused: the anonymized text stays local";
64   end process;
65
66   process(anonymizer.failed, frontier.failed)
67   begin
68       status <= "a model call failed";
69   end process;
70 end hybrid;

```

1. ローカル LLM (anonymizer) が records を匿名化して draft (restricted) にします。
2. release が、PII 検査と人の承認の両方で認めたときだけ、draft を anonymous (internal) に解除します。
3. クラウドの frontier が anonymous と question を解析します。
4. 解除が拒否されると release.failed で理由を残し、何もクラウドへ出ません。

このサンプルを少し崩すと、検査がそれぞれ止めます。

崩し方	診断
declassify を使わず <code>anonymous <= draft;</code> と書く	E303
<code>approval => human</code> を外し、フィルタだけにする	E304
匿名化前の draft をクラウドへ渡す	E301、E302、E303

フィルタだけの declassifier は内容しか見ないため、restricted の事象で起動された process の中では使えません (E304)。人の承認がある declassifier は、承認依頼に解除の文脈も示すので、from までの文脈で使えます。

8.5 設計の外にある安全機構

次の規則は設計ではなく、AI Conductor の設定とランタイムが強制します。

- **capability** : どの route がどのツール・ファイル・モデルを使えるか。設定は config/

capabilities.toml にあります。

- **Effect と承認**：書き込みを伴う呼び出しの記録と、外部への書き込み前の人間の承認。
- **監査**：実行ごとの `.runtime/runs/<id>/events.jsonl`。

設計から device を呼び出すと、これらがすべて適用されます。設計の側からこれらが無効にする構文はありません。

9 設計を図にする (aic graph)

aic graph は、aic check に合格した設計を図にします。図は検査済みの設計から導いた派生ビューであり、実行の意味は定めません（実行の意味は `RUNTIME_SPEC` が定めます）。検査に通らない設計は描きません。

9.1 ビューと出力形式

表 2 aic graph のビュー (--view)

ビュー	表すもの
structure	entity の境界、ポート、signal、配置ごとの zone に入った device、主なデータの流れ
behavior	process と、それを起動する事象、非同期の呼び出し、分岐、parallel、バリア、timeout、ループ
activity	UML アクティビティ図。process とバリアごとに 1 つのアクティビティ
petri	データを抽象化した Petri ネット (P/T ネット)。解析ツールに渡せる
security	機密区分、配置の zone、クラウドへ出る流れ、人の承認、拒否される流れ
state	宣言された状態。v0.1 には状態型がないため、現在は常に拒否される

表 3 出力形式 (--format)

形式	使い方
mermaid	既定。GitHub などの Markdown に貼るとそのまま描かれる
dot	Graphviz で PDF・SVG・PNG にする (dot -Tpdf など)。本節の図はこの形式
json	図の中身 (ノード・辺・グループ) そのもの。ほかのツールで使う
plantuml	activity ビュー専用。PlantUML で UML のアクティビティ図にする
pnml	petri ビュー専用。Petri ネットの標準形式 PNML (ISO/IEC 15909-2)

```
# Graphviz で PDF にする
aic graph examples/tutorial/05_parallel_barrier.awhdl \
  --view behavior --format dot | dot -Tpdf -o behavior.pdf

# PlantUML でアクティビティ図にする
aic graph examples/tutorial/05_parallel_barrier.awhdl \
  --view activity --format plantuml --output activity.puml
```

```
java -jar plantuml.jar activity.puml
```

```
# ラベルを 1 行に縮める / 機密区分や route も表示する
aic graph ... --compact
aic graph ... --show-classification --show-capabilities
```

本節の図は docs/tutorial/figures/build.sh で作り直せます。

表 4 aic graph の終了状態

状態	意味
0	成功
1	構文解析に失敗
2	構造または Profile の検査に失敗
3	情報フローの検査だけに失敗
4	図にできない (state ビュー、architecture が複数で選ばれていない: G401)
5	指定した形式でそのビューを表せない (例: security ビューを PNML で: G501)
6	ファイルを読めない、または書けない

9.2 構成を見る: structure ビュー

図 1 は examples/tutorial/02_prolog_verdict.awhdl の構成です。ポートは entity の枠の外、signal と device は中に置かれ、device は配置ごとの zone (ここでは local) に入ります。実線はデータの流れ、破線は事象による起動です。rules の失敗 (failed) が verdict を書く process を起動することも読み取れます。

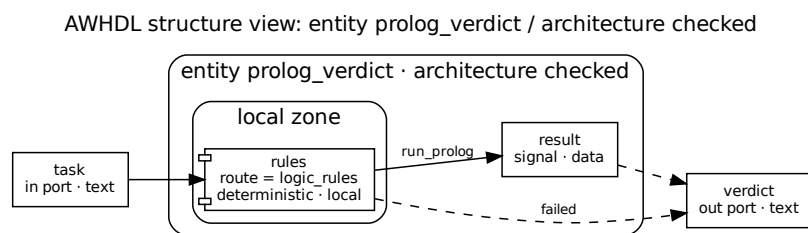


図 1 structure ビュー (examples/tutorial/02_prolog_verdict.awhdl、--show-capabilities)

9.3 振る舞いを見る: behavior ビュー

図 2 は examples/tutorial/05_parallel_barrier.awhdl の振る舞いです。

- 細い角丸の四角が process、太い角丸の四角が device の呼び出し、四角が signal、旗の形が device の事象 (prover.failed) です。

- 破線は非同期の関係（事象による起動、呼び出しの完了）、実線は process の中の制御です。
- 六角形の `parallel` から分かれた呼び出しは、`commit together` でまとめて反映されます。この合流とバリアは、同じ実行・同じ世代の結果だけをそろえます。
- `prover.failed` のように、どこかの process が待っている事象だけを描きます。待たれていない事象も描くには `--show-internal` を付けます。

AWHDL behavior view: entity `two_checks` / architecture joined

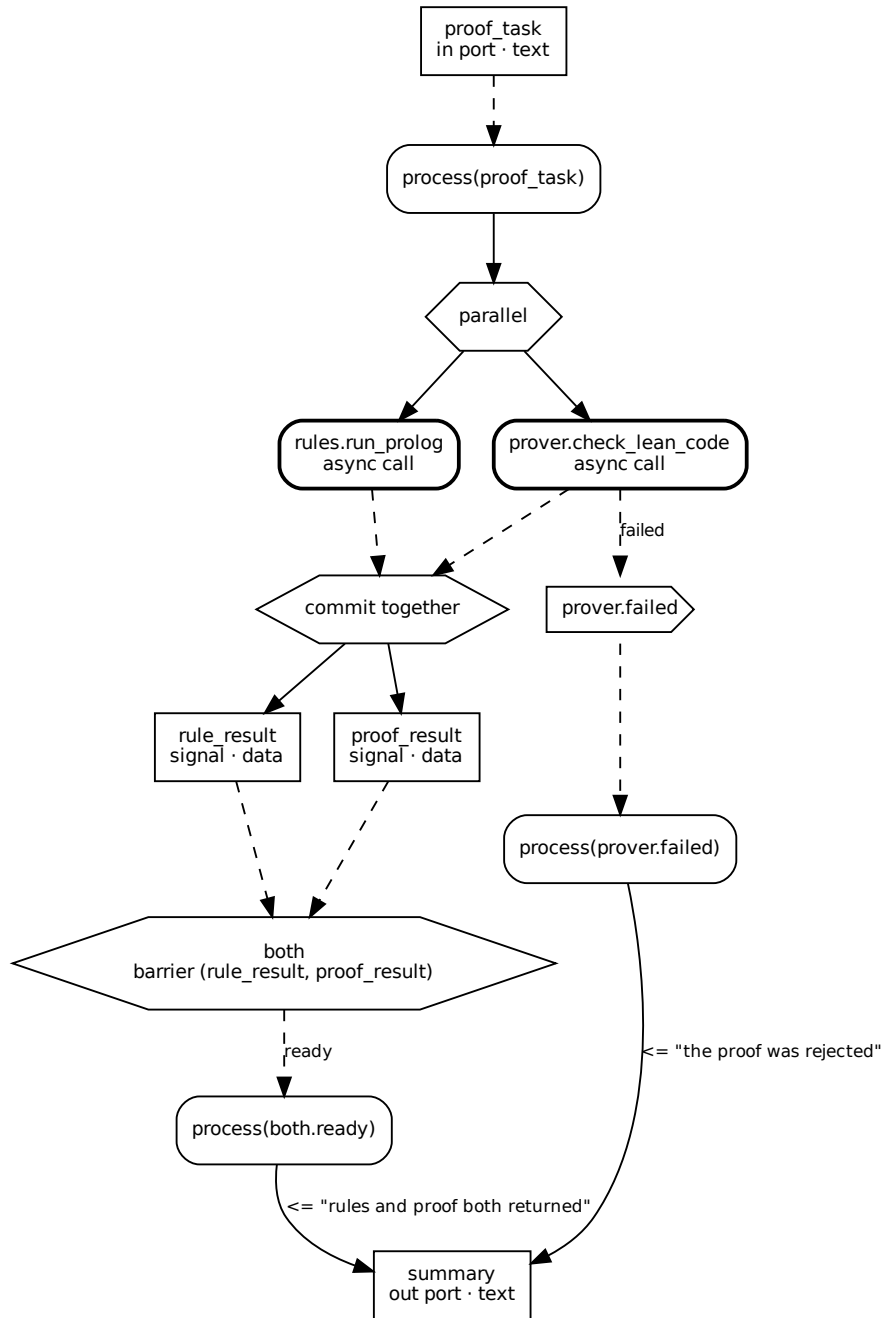


図 2 behavior ビュー (`examples/tutorial/05_parallel_barrier.awhdl`)

フィードバックループ (examples/tutorial/04_feedback_loop.awhdl) では、ループを閉じる辺に loop と表示されます。

9.4 UML アクティビティ図：activity ビュー

図 3 は同じ設計のアクティビティ図です。process ごとに 1 つのアクティビティがあり、開始（黒丸）→ 起動する事象の受信（左が尖った五角形）→ アクション（角丸）→ 事象の送信（右が尖った五角形）→ 終了（二重丸）と進みます。parallel は太い棒の fork と join に、呼び出しの成否はひし形の分岐になります。バリアも 1 つのアクティビティで、メンバーの変化をすべて受け取ると both.ready を送ります。

アクティビティ同士は線で結ばれません。UML と同じく、送信と受信のシグナル名でつながります。たとえば右端のバリアが送る both.ready を、2 番目のアクティビティが受け取ります。

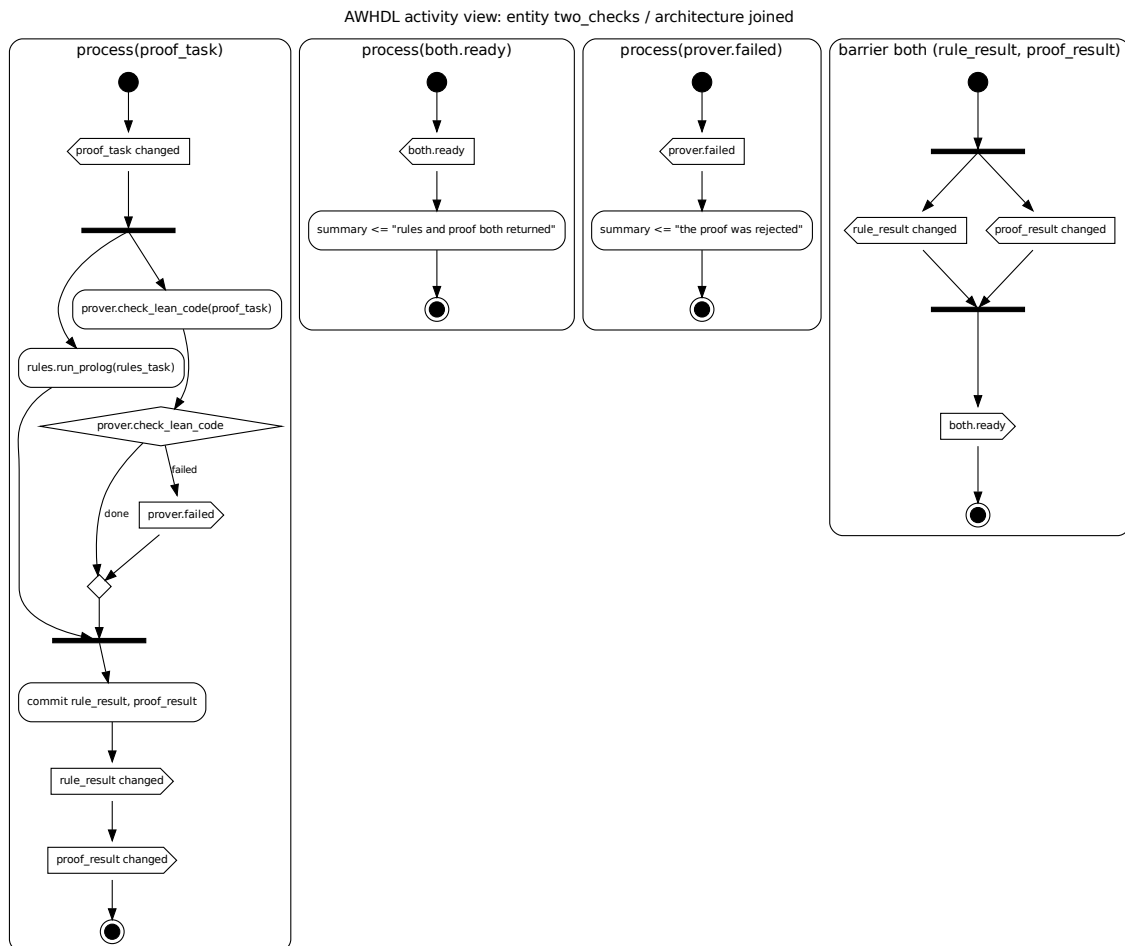


図 3 activity ビュー (examples/tutorial/05_parallel_barrier.awhdl)

9.5 Petri ネット：petri ビュー

図 4 は `examples/tutorial/04_feedback_loop.awhdl` を Petri ネットにしたものです。丸が place（状態）、四角が transition（動作）で、線は必ず丸と四角を結びます。

- 黒い点はトークンです。入力ポート `code` で起動する process の place (`p1 woken`) にだけ、最初からトークンがあります。
- `round` を変える書き込みから `p2 woken` へ戻る破線が、フィードバックループです。
- 分岐の条件 (`[round >= 1 and round <= 3]`) は評価せず、どちらの分岐も起こり得るものとして描きます。書き込みも「値が変わった (`changed`)」と「変わらなかった (`unchanged`)」に分かれます。
- 呼び出しは成功・失敗・timeout の 3 つの transition が競合します。

データを抽象化しているため、設計の実際の実行はどれもこのネットの発火列になりますが、逆は成り立ちません。「この状態に到達し得るか」を調べる安全性の解析には使えますが、停止性の解析には使えません。--format pnml で出力すると、Petri ネットの解析ツールで読み込めます。

9.6 情報フローを見る：security ビュー

図 5 は、公式サンプル `examples/secure_cooperation.awhdl` (第 8.4 節) の情報フローです。signal は機密区分ごとの枠にまとめられます。

- 紫の太線 (declassified) が機密解除です。区分が下がるのはこの辺だけで、`release` が `draft` (`restricted`) を `anonymous` (`internal`) にします。解除の前に `draft` が PII 検査 (`pii_check`) に渡ることも描かれます。
- 太線はクラウドへ出る流れです。クラウドの `frontier` に届くのは `anonymous` (`internal`) と `question` (`public`) だけです。
- 赤い点線 (DENIED) は拒否される流れです。`restricted` の区分はすべてクラウドへ送れません。区分の枠全体が拒否されるときは、枠から 1 本にまとめて描きます。
- 拒否や解除は色だけでなく線の種類と文字でも示すので、白黒で印刷しても区別できます。

図 6 は同じサンプルの Petri ネットです。activity・petri・behavior のビューでは、device の location からレーンを描き、保護領域 (`local` (`protected`), 緑) と外部領域 (`cloud` (`external`), 青) の境界を示します。クラウドで起きるのは `frontier.analyze` の結果だけで、境界を越える線は太線になります。機密解除は、紫の `granted` の transition と `refused` の transition の競合として描かれます。レーンを描かないときは `--no-lanes` を付けます。

第 8 節の `examples/tutorial/06_information_flow.awhdl` は検査に通らないため、図になりません。

```
$ aic graph examples/tutorial/06_information_flow.awhdl --view security
...:14:9: AWHDL-E301: Restricted data cannot flow to cloud device reviewer
...: not drawn: the design does not pass aic check
$ echo $?
3
```

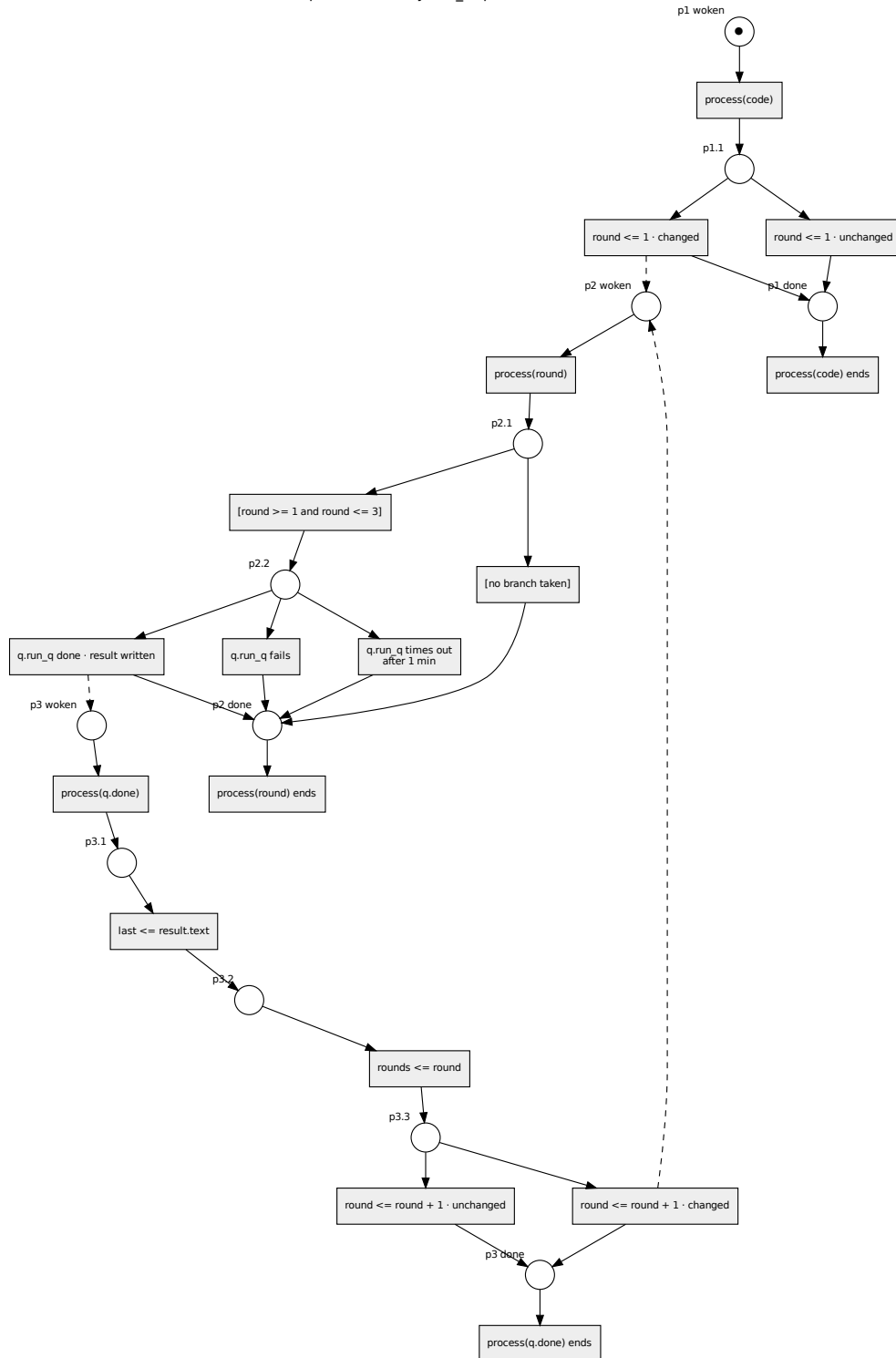
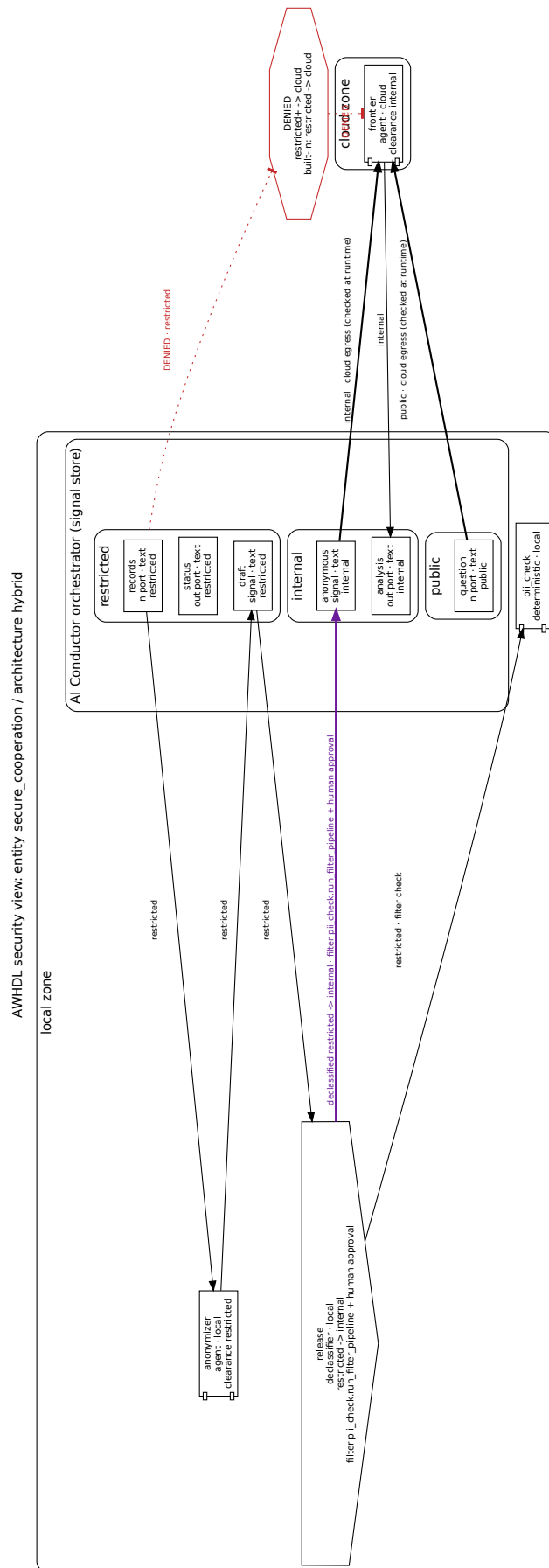


図 4 petri ビュー (examples/tutorial/04_feedback_loop.awhdl)



☒ 5 security ビュー - (examples/secure_cooperation.awhdl)

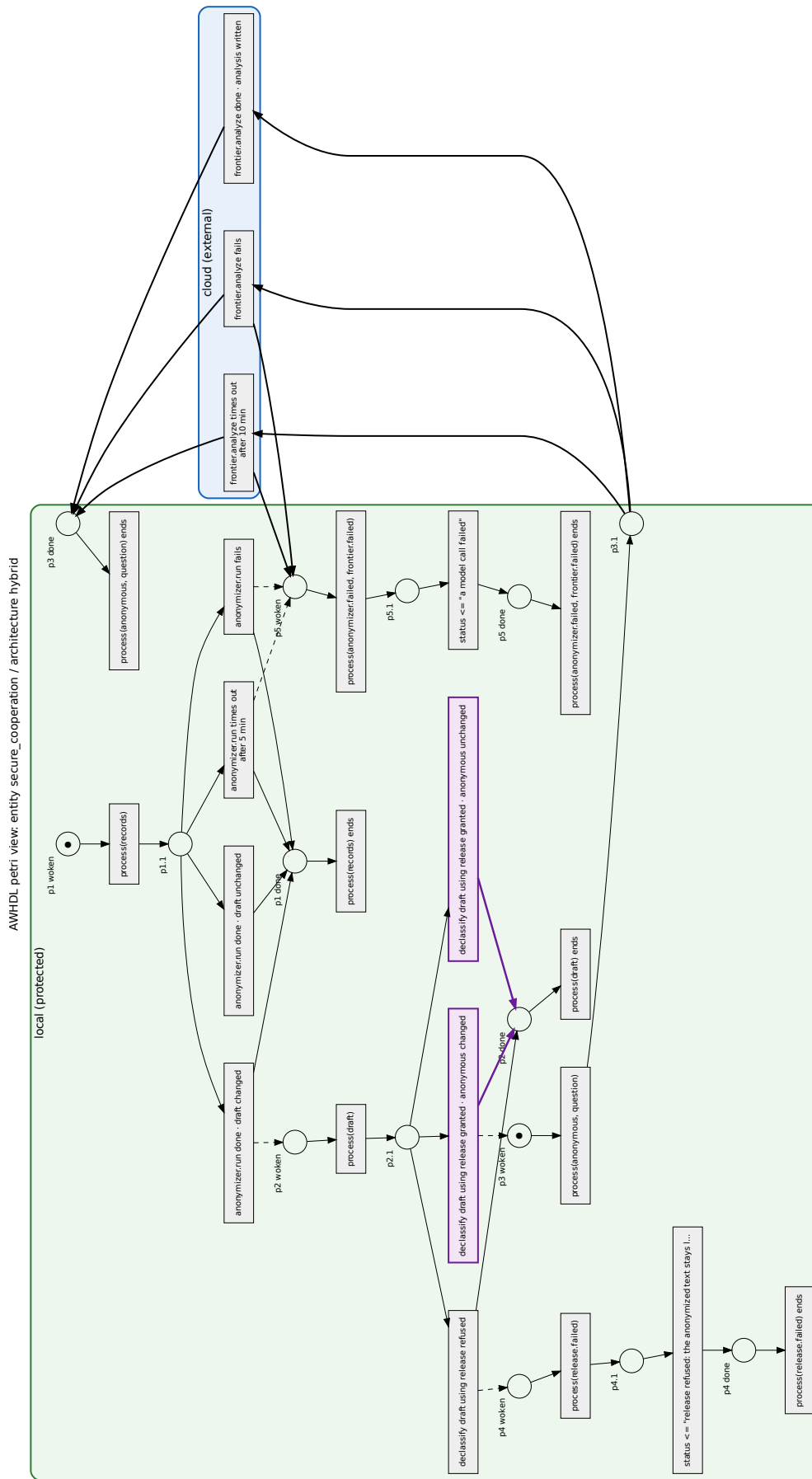


Fig 6 petri net (examples/secure_cooperation.awhdl)

図は設計の説明であって、規則ではない

図は検査済みの設計から機械的に作られますが、実行の意味を決めるのは `RUNTIME_SPEC` です。特に Petri ネットはデータを抽象化した近似です。図だけを根拠に「この設計は必ず止まる」などと判断しないでください。

10 device と route

10.1 束縛の規則

表 5 device の種類と結び付けられる route

device の種類	結び付けられる route	例
agent	モデルの route、または Codex	<code>initial_coding</code> 、 <code>deep_reasoning</code>
mcp	MCP の route (Codex 以外)	<code>calculation_graphing</code> 、 <code>table_analysis</code>
deterministic	読み取り専用の MCP の route	<code>logic_rules</code> 、 <code>logic_proof</code> 、 <code>text_filtering</code>

主制御 (planner) の route (loop) には結び付けられません。

10.2 AI Conductor の route (2026 年 9 月時点)

route	種類	実行先
<code>initial_coding</code>	モデル	192.168.6.10 の RTX 4090 (Qwen3.8 27B)
<code>japanese_polishing</code>	モデル	192.168.6.10 (llm-jp-4 8B)
<code>vision</code>	モデル	192.168.6.20 の RX 9060 XT (Qwen3-VL 8B)
<code>logic_rules</code>	MCP (読み取り)	192.168.6.10 の SWI-Prolog
<code>logic_proof</code>	MCP (読み取り)	192.168.6.10 の Lean 4.33.1
<code>text_filtering</code>	MCP (読み取り)	ローカルの制限付き Filter
<code>table_analysis</code>	MCP (書き込み)	192.168.6.10 の KDB/q
<code>calculation_graphing</code>	MCP (書き込み)	192.168.6.10 の MATLAB R2026a
<code>deep_reasoning</code>	Codex (cloud)	Codex CLI (読み取り専用)
<code>open_data_acquisition</code>	Codex (cloud)	Codex CLI (承認が必要な外部書き込み)

10.3 呼び出しと引数

- 引数が 1 つの文字列なら、それが device への入力になります。
- MCP の route では、メソッド名がその route のツール名と一致すれば、そのツールを使います (例: `rules.run_prolog`、`prover.check_lean_code`、`q.run_q`)。
- ツールの引数はアダプタが入力から取り出します。Prolog はバッククォートの節と問い合わせ、Lean は宣言 (`theorem` など)、KDB は `q` のコード、Filter はプロジェクト内のパスです。

- MATLAB と KDB では、日本語の文章をコードとして送ることはせず、呼び出しを失敗させます。

11 v0.1 で使えないもの

次の構文は v0.1 に含まれず、構文エラーとして拒否されます。黙って受理されることはありません。

- case、await、type 宣言と状態機械、retry、approve、policy、export、configuration、時相アサーション
- Event 宣言と完了条件 (completion when hard {...}) の構文。意味論は実装済みで、完了条件は現在 config/routing-defaults.toml で与えます。
- 機密区分 confidential / secret、配置 sandbox / private_cloud / external (E401)

付録 A 文法の早見表

```

1  entity name is
2      port ( a : in text<internal>; b : out data<public> );
3  end name;
4
5  architecture arch of name is
6      device rel : declassifier
7          generic (from => restricted, to => internal,
8                  filter => d, filter_method => check, approval => human);
9      device d : agent | mcp | deterministic
10         generic (route => "<route>", location => local | cloud,
11                 clearance => public | internal | restricted);
12     signal s1, s2 : data<internal> := 0;
13     timer t : period 1 sec;
14     budget lim is iterations <= 10; wall_time <= 5 min;
15         tool_calls <= 3; model_calls <= 2; end budget;
16     barrier bar (s1, s2);
17 begin
18     assert always (<expr>);
19     assert never (<expr>);
20     assert never (internal -> cloud);
21
22     process(a, s1.changed, d.done, d.failed, d.timeout, t, bar.ready)
23         timeout 2 min;
24     begin
25         d.method(<expr>) timeout 30 sec -> s1;
26         s1 <= declassify s2 using rel;
27         s2 <= s1 + 1;
28         if <expr> then null; elsif <expr> then null; else null; end if;
29         parallel d.m1(a) -> s1; d.m2(a) -> s2; end parallel;
30         assert <expr>;
31     on timeout

```

```

32         b <= "timed out";
33     end process;
34 end arch;

```

式の結合は弱い順に or、and、not、比較 (=、/=、<、<=、>、>=)、整数の + / - です。比較は数値どうし、または文字列どうしで行います。

付録 B 診断コード

コード	意味
E001	ソースファイルを読み取れない
E101	構文エラー
E201–E205	重複した entity・ポート・device・宣言、未知の entity
E206	感度リストの未知の名前、または runtime が発生させない事象の名前 (d.completed など)
E207 / E208 / E209	未知の device、未知の出力先
E210	式の中の未知の名前
E211	正でない時間 (タイマー周期、制限時間)
E212	不正または重複した予算項目
E214	signal でないバリアのメンバー
E215	2 つの parallel 分岐が同じ signal に書き込む
E217	入力ポートへの書き込み
E218	device の generic の重複や型の誤り
E219	declassifier の宣言の誤り (範囲、手段、配置)
E220	declassifier でない device での解除、または declassifier の直接の呼び出し
E301	クラウドの device へ流せないデータ
E302	device の clearance を超えるデータ
E303	高い区分のデータを低い区分の signal に保存
E304	暗黙のフロー：文脈の区分が書き込み先より強い
E305	暗黙のフロー：cloud の下限以上の文脈でクラウドの device を呼ぶ
E306	暗黙のフロー：文脈の区分が device の clearance を超える
E307	解除するデータの区分が declassifier の from を超える
E401	v0.1 の Profile に含まれない構成要素
E402	未知の名前 (区分、配置など)
G401	図にできない (aic graph : state ビュー、architecture の曖昧さ)
G501	指定した形式でそのビューを表せない (aic graph)

付録 C 参考文献

- `awhdl/docs/PROFILE_v0.1_ja.md` : v0.1 の範囲と実装状況 (正本)
- `awhdl/docs/LANGUAGE_SPEC_ja.md`、`AWHDL_LANGUAGE_SPEC_v0.2_ja.md` : 言語仕様
- `awhdl/docs/RUNTIME_SPEC_ja.md` : delta cycle、Effect、完了評価
- `awhdl/docs/SECURITY_SPEC_ja.md` : 機密区分、capability、承認
- `awhdl/docs/GETTING_STARTED_ja.md` : 最初の一步
- `awhdl/docs/GRAPH_ja.md` : aic graph の使い方